CS 230 Course Notes: Computer Systems

Winter 2026 - Sandy Graham

Talha Yildirim,
tyildir [at] uwaterloo [dot] ca

Contents

1 Assembly Language 3

1 Assembly Language

Definition 1.1 (Machine Code)

- Binary code - comprised of 0's and 1's
- "Direct" execution by processor
- Program grouped into instructions
  - fixed v.s. variable length
  - operation code (opcode) + operands
  - Instructions control processor
    - opcode designates operation
    - operands designate data

♣

Definition 1.2 (Assembly Language)

- Human readable "programming language"
  - Very simple compared to Racket, Python, etc
- Almost direct mapping to machine code
  - Except a few concepts
- Assembler turns it into machine code
  - Process is called "assembling" rather than compiling

♣

Definition 1.3 (Instruction Set)

An instruction set is the complete list of machine-level commands a CPU knows how to execute.

- Repertoire of instructions
- Different processors have different sets
  - Many commonalities
    - mathematical
    - memory access
    - control flow

♣

Definition 1.4 (Turing Completeness)

- Fundamental capabilities of instruction set
- Minimum requirements
  - Condition branching (if / goto)
  - Some form of repetition (while / for / recursion)
  - Arbitrary memory manipulation
- Typical programming languages / instruction sets are Turing complete

♣

Definition 1.5 (MIPS Architecture)

- MIPS: Microprocessor without Interlocked Pipeline Stages
- A type of computer architecture
- Multiple revisions, systems, and compilers
  - Not just a single standard MIPS

♣

Definition 1.6 (MIPS Assembly Language)

- Each instruction takes 32 bits
  - 4 bytes = 1 word
- Arithmetic instructions operate on registers
  - 32 registers available number $0 to $31
  - Register $0 always equals 0
- Instructions have up to 3 operands
  - 1st is destination, 2nd and 3rd are sources
  - Same register can be source and destination
- Two addressing modes
  - **Register:** Operands in registers
  - **Immediate:** One operands is a 16-bit constant
- Memory: 32-bit address space
  - Relative and absolute memory load / store
  - Relative and absolute branch instructions

♣

Definition 1.7 (Special Registers)

- **PC**
  - Program counter location (byte address) in machine code
    - incremented by 4 for each instruction
- $0
  - Constant 0
- Register Conventions
  - $29 - stack pointer ($\sp)
  - $30 - frame pointer ($\fp)
  - $31 - return address ($\ra)

♣

Definition 1.8 (Addition)

```
add $3, $2, $11
```

- Add (contents of) register `1` and `2`
- Place result in register `3`
- General notation: `add $d, $s, $t`
- Move (contents of) register `1` to `2`: `add $2, $1, $0`

```
sub $d, $s, $t
```

- Subtract register `t` from `s`
- Place result in register `d`

♣

Definition 1.9 (Debugging)

- **Use output:**
  - Print intermediate values as your program runs so you can see what's happening
- **Early Termination with `jr $31`:**
  - `$31` (also called `$ra`) is the return address register. Executing `jr $31` effectively returns from your program early — it stops execution at that point. The trick is:
  - Insert `jr $31` partway through your code
  - Run the program and inspect the register values at that stopping point
  - Verify each register holds what you expect
  - Move `jr $31` further down and repeat
- General Techniques
  - **Analyze Log Outputs:**
    - Run your program and carefully read any output or error logs the emulator produces. The emulator may show you register states or flag unexpected behavior
  - **Controlled step-by-step execution:**
    - Rather than running the whole program at once, execute it instruction by instruction and check the state after each one. This is what a proper debugger does automatically, but in this environment you simulate it manually using the early termination trick above.

♣

Definition 1.10 (Immediate Addition)

```
addi $t, $s, i
```

- Adds registers `s` and value `i`
- value `i` can be negative - size / range?
- Place results in register `t`
- Use to initialize registers: `addi $t, $0, i`

♣

Definition 1.11 (Multiplication and Division)

Special registers `hi`, `lo`

```
mult $s, $t
```

- Multiply registers `s` and `t`
- Place results in `hi:lo`

```
div $s, $t
```

- Divide register `s` by `t` (integer division)
- Place result in `lo`, remainder in `hi`

```
mfhi $d
```

- Copy contents of `hi` to `$d`

```
mflo $d
```

- Copy contents of `lo` to `$d`

♣

Problem.

Compute average of three numbers

- Values in `$3, $4, $5`
- Result in `$2`

Solution.

```
add $2, $3, $4

add $2, $2, $5

addi $1, $0, 3

div $2, $1

mflo $2
```

Problem.

Compute $(a + b) - (c + d)$

- a, b, c, d in `$1, $2, $3, $4` - preserve values
- results in `$5`

Solution.

```
add $5, $1, $2

add $6, $3, $4

sub $5, $5, $6
```

Definition 1.12 (Conditional Execution)

- Computation dependent on intermediate results
- Otherwise only linear number crunching

♣

Definition 1.13 (Conditional Branching)

`beq $s, $t, i`

- Compare registers `s` and `t`
- `if` equal. skip `i` instructions
- `i` can be negative

`bne $s, $t, i`

- compare registers `s` and `t`
- `if` not equal, skip `i` instructions
- `i` can be negative

The `PC` (Program Counter) The `PC` is a register that holds the address of the next instruction to execute. Normally it just increments by 4 each cycle (since each instruction is 4 bytes).

`if ($s == $t) PC += i *4`

- `PC` increases automatically by 4 when the instruction is fetched and before it is executed
- execution of instruction results in `PC = PC + i * 4`

♣

Definition 1.14 (Set Less Than)

`slt $d, $s, $t`

- compare register `t` and `s`
- `$s < $t` $\rightarrow$ `$d = 1`
- `$s >= $t` $\rightarrow$ `$d = 0`

♣

Example – Loop

Example PC Value	Assembly Instruction
00	`addi $2, $0, 1`
04	`addi $1, $0, 10`
08	`mult $2, $1`
0C	`mflo $2`
10	`addi $1, $1, -1`
14	`bne $1, $0, -4`

Definition 1.15 (Branch Labels)

The Problem Without Labels

```
bne $1, $0, -4
```

That `-4` means "jump back 4 instructions." But what if you add or remove instructions inside the loop? You'd have to manually recalculate that number every time. That's error-prone and annoying.

The Solution: Labels

Instead of writing a raw offset number, you write a named label and let the assembler figure out the offset for you.

```
addi $1, $0, 10      ; $1 = 10

loop:                ; <-- this is just a named marker, not an instruction
  addi $1, $1, -1  ; $1 = $1 - 1
  bne $1, $0, loop ; if $1 != 0, jump back to "loop"
```

The label loop: marks a position in the code. When the assembler sees `bne $1, $0, loop`, it automatically calculates the correct offset number needed to jump back to that position.

♣

Definition 1.16 (Conditional Execution)

This is how you write an `if/else` statement in assembly. There's no `if` keyword — you have to manually redirect the `PC` using branches and labels.

```
; compute something
beq $1, $2, eqcase     ; if $1 == $2, jump to eqcase
                        ; otherwise fall through to the next line

; do something for $1 != $2   <-- the ELSE branch
beq $0, $0, final      ; unconditional jump to final (skip the if-branch)

eqcase:
  ; do something for $1 == $2   <-- the IF branch

final:
  ; continue with program
```

♣

Definition 1.17 (Constants)

Method 1: `addi` (Add Immediate)

```
addi $4, $0, 100
```

This adds `100` to register `$0` (which is always `0`) and stores the result in `$4` — effectively loading the constant `100` into `$4`.

Method 2: `lis` (Load Immediate and Skip)

```
lis $4
.word 0xA3257CE2
```

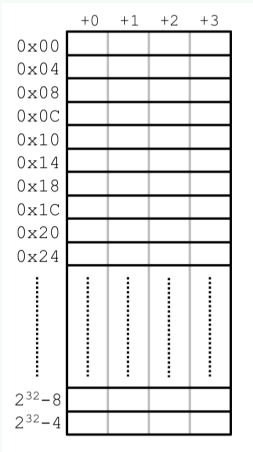
`lis` solves the `16`-bit limitation by using a clever trick:

1. It looks at the very next word in memory (the next `32` bits after the instruction)
2. Loads that full `32`-bit value into a register
3. Then skips over it (so the CPU doesn't try to execute that data as an instruction)

♣

Definition 1.18 (Memory Model)

- **Byte Addressable**
  - Memory is organized as a giant array of bytes, where every single byte has its own unique address
  - With `32`-bit addresses, you can have $2^{32} = 4,294,967,296$ unique addresses
  - Each address points to exactly 1 byte of data
- **Word Aligned**
  - A word in this architecture is 4 bytes (32 bits). While individual bytes have addresses, you can only load or store a full word at a time, and only at specific addresses
- **Unaligned Access Errors**
  - If your program tries to read or write a word from an address that is not a multiple of 4 (e.g., address `0x03`), the CPU cannot do it and throws an "unaligned access" error — the program crashes.



- **The big takeaway:**
  - You address bytes individually, but you read/write 4 bytes (a word) at a time, and that word must start at an address divisible by 4

♣

Definition 1.19 (Memory Access)

`lw $t, i($s)`

- Load word from location `s + i` into `t`
- `s + i` must be word aligned

`sw $t, i($s)`

- store word from `t` into locations `s + i`
- `s + i` must be word aligned

`s + i`

- The memory address to read/write
- `$s` is a register holding a base address (your starting point in memory)
- `i` is a constant offset (how many bytes to shift from that base)
- The CPU adds them together to get the final address

♣

Definition 1.20 (Memory Addresses)

1. **Register Number**

- This is just which register you're referring to — a number between 0 and 31
- `$4, $20, $31` ← these are register numbers

2. **Register Contents**

- This is the actual value stored inside that register — a `32`-bit number
- `$20` might contain the value `0x100`

3. **Memory Address**

- When you write `lw $t, i($s)`, the value `$s + i` is a memory address — it's the location in memory you want to access
- `$s = 0x100, i = 8` memory address = `0x108` ← this is a location, not data

♣

Definition 1.21 (Structures)

A struct is a group of related fields stored sequentially in memory, each `4` bytes apart. If `$s` holds the base address (start of the struct), you access each field using offsets:

```
lw $t, 0($s)   → year        (base + 0)
lw $t, 4($s)   → month       (base + 4)
lw $t, 8($s)   → day         (base + 8)
lw $t, 12($s)  → city        (base + 12)
```

The offsets are always `4` bytes apart because each field is one word (`4` bytes)

Key idea:

`$s` is set once to the start of the record. You never change it — just change the offset `i` to navigate to different fields.

♣